



B.Sc. Software Engineering — Graduation Project Defense

Req2UI

From Natural Language to Secure Software Artifacts Using AI

Arab Academy for Science, Technology & Maritime Transport (AASTMT) — College of
Computing and Information Technology, Smart Village — 2025/2026

TEAM Mennatallah Hany · Mayar Hany · Yasmin Haitham · Malak Tarek · Mohamed Bassem

SUPERVISOR Dr. Salma Radwan Abdel-Hamid

What we'll cover

Eight sections, ~20 minutes.

01

**Motivation &
Problem**

Why manual SRS fails.

02

**Objectives & Related
Work**

What we set out to do.

03

**Solution &
Architecture**

The big idea.

04

**The 10-Stage
Pipeline**

Centerpiece.

05

**Concurrency &
Security**

Fast & safe.

06

**UML, UI &
Traceability**

The outputs.

07

Tech Stack & Demo

What's running.

08

**Testing & Future
Work**

Validation + roadmap.

Why Req2UI?

Manual SRS authoring fails the modern team.

01

Slow

Requirements authored by hand take days to weeks per project. Iteration is painful and feedback loops are long — especially when stakeholders pivot mid-cycle.

02

Inconsistent

Style, depth, and terminology drift across teams and projects. Two engineers reading the same input produce two very different SRS documents.

03

Insecure

Security requirements are routinely missing or bolted on late — treated as an afterthought rather than a first-class deliverable of the spec phase.

The tooling gap

Today's tools each cover only one slice — none spans the whole flow.

A

Requirements tools

Generate an SRS document and stop there. No downstream artifacts, no security generation, no traceability to tests or UI.

B

UI prototyping tools

Produce screens and components, but assume requirements are already settled upstream. No link back to a spec.

C

Test automation tools

Assume requirements exist. They generate tests from an external spec or from code — not from natural language.

No tool auto-generates security requirements. And no tool keeps end-to-end traceability across artifacts.

Five objectives

What Req2UI set out to deliver.

01

Single input

A plain-language description of a software system — no structured forms, no DSL.

02

Full SRS

A complete IEEE 830 SRS: functional, non-functional & security requirements.

03

Test cases

Functional tests (IEEE 829) + security tests driven by OWASP Top 10 (2021).

04

UML + UI

UML diagrams (use-case, class, sequence) + runnable HTML/Tailwind per screen.

05

Traceability + export

End-to-end traceability + export as PDF, DOCX, LaTeX & CSV.

What exists, what's missing

Each category covers one slice. Req2UI spans all five.

CATEGORY	REQUIREMENTS	UI PROTOTYPE	TESTS	SECURITY	TRACEABILITY
Requirements generators	✓	×	×	×	×
UI prototypers	×	✓	×	×	×
Test automators	×	×	✓	×	×
Security scanners	×	×	partial	✓	×
Req2UI ours	✓	✓	✓	✓	✓

First integrated NL → SRS → UI → functional + security tests pipeline, with auto security generation and full traceability.

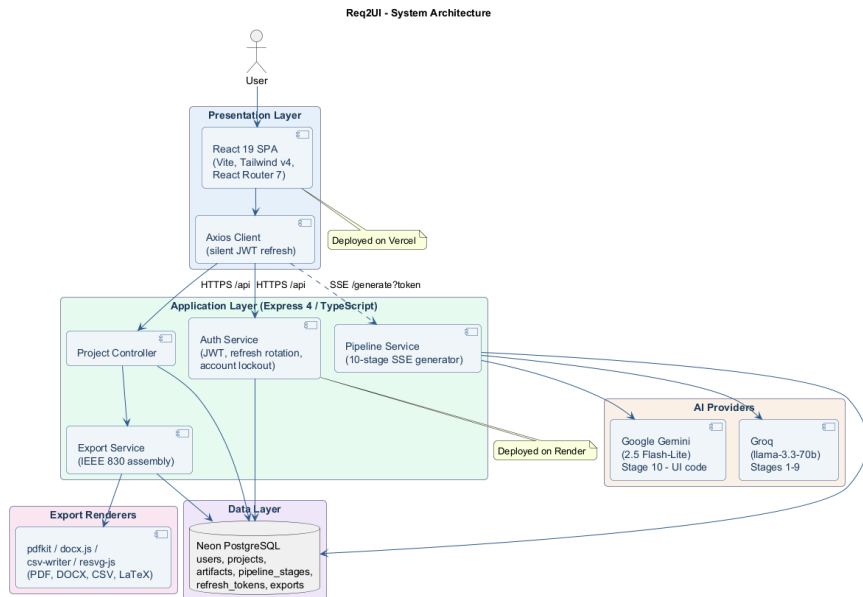
One input. A complete, security-aware suite.

A single natural-language description drives a 10-stage AI pipeline.



Layered architecture

React SPA → Express API → Groq / Gemini → Neon Postgres.



PRESENTATION

React 19 SPA

TypeScript app — Tailwind v4, Vite, React Router 7.
An SSE listener streams live pipeline progress.

APPLICATION

Express 4 API

Orchestrates the 10-stage pipeline. JWT auth, Zod validation, Helmet, rate limiting.

DATA

PostgreSQL on Neon

The 10-stage pipeline

From natural language to runnable artifacts.

01**Requirement****Extraction**

Parse NL; emit IEEE 830 narrative.

02**Functional Reqs**

IEEE 830 "the system shall...".

03**Non-Functional****Reqs**

Performance, usability, reliability + metrics.

04**Security Reqs**

Mapped to OWASP Top 10 (2021).

05**Functional Tests**

IEEE 829 structured cases.

06**Security Tests**

Attack-vector driven, OWASP-aligned.

07**UI Wireframes**

Per-screen layout & components.

08**Traceability Matrix**

Req ↔ test ↔ security, coverage %.

09**UML Diagrams**

Use-case, class, sequence (Mermaid).

10**Runnable UI Code**

HTML + Tailwind, two-pass.

↳ Sequential 1-2 · parallel 3-5 · 6-7 · 8-10

◆ Stages 1-9 · Groq Llama-3.3-70b

◇ Stage 10 · Gemini 2.5-flash-lite

Two LLMs, one pipeline

Each model owns what it's best at.

Groq

LLAMA-3.3-70B-VERSATILE

- Drives **stages 1–9** — narrative, requirements, tests, traceability, UML.
- Fast inference on Groq's LPU keeps the pipeline responsive.
- Structured JSON output for reliable downstream parsing.
- Cost-efficient for high-volume text generation.

Google Gemini

GEMINI-2.5-FLASH-LITE

- Drives **stage 10** — runnable UI code generation.
- Two-pass: a shared design system, then one HTML page per screen.
- Strong code generation at a fraction of a large model's cost.
- Produces Tailwind-styled, runnable HTML out of the box.

A pipeline router dispatches each stage to its assigned model — no manual handoff.

Parallel stages, resumable runs

Independent stages run concurrently; failures pick up where they stopped.

LANE A · SEQUENTIAL

01



02

LANE B · PARALLEL (AFTER 02)

03

04

05

LANE C · PARALLEL (AFTER 05)

06

07

LANE D · PARALLEL (AFTER 07)

08

09

10

Persisted artifacts

Every artifact saved keyed by (project, type).

Resume on failure

Interrupted runs pick up from the last saved stage — no re-do.

Live progress

Status pushed to the browser in real time via SSE.

Security, not bolted on

OWASP Top 10 requirements generated in stage 4; the auth stack hardened end-to-end.

Stage 4

Auto-generates security requirements aligned to OWASP Top 10 (2021).

JWT access tokens

15-minute TTL; signed & verified per request.

Rotating refresh tokens

7-day TTL, SHA-256 hashed, family theft detection.

bcrypt hashing

Password hashing at cost factor 12.

Account logout

Locks for 15 min after 5 failed logins.

Helmet + CORS

Security headers + strict CORS allow-list.

Zod validation

Every API route validates its payload.

Rate limiting

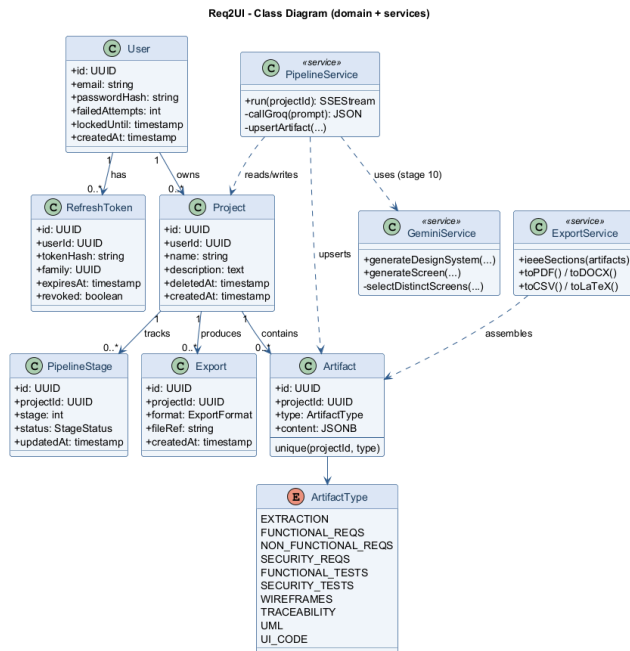
Auth (5/15min) + generation (10/hr) throttled.

Secrets in env

API keys in env vars, never in source.

Diagrams and runnable UI

Mermaid → PNG for UML; HTML + Tailwind per screen for UI.



UML — USE-CASE · CLASS · SEQUENCE

Diagrams are emitted as **Mermaid** — editable, version-controllable text — then rasterized to PNG via resvg-js for embedding in the PDF export.

UI CODE (STAGE 10) — TWO-PASS

Pass 1 generates a shared design system (colors, type, spacing, components). **Pass 2** emits one HTML + Tailwind page per distinct screen, all consuming it — a consistent, runnable prototype.

Every artifact links to every other

An illustrative matrix with coverage %, plus four export targets.

REQUIREMENT	TEST CASE	SECURITY	STATUS
FR-01 User login	TC-001	—	✓
SR-01 OWASP A07	—	STC-005	✓
FR-04 Export PDF	TC-018	—	✓
NFR-02 Response < 500ms	TC-022	—	✓
SR-03 OWASP A03	—	STC-009	✓

PDF

Stakeholder-ready bundle.

DOCX

Editable for downstream revision.

LaTeX

Academic / publication-ready.

CSV

Machine-readable matrix.

Example coverage: each FR maps to a test case; each SR to a security test. Real coverage is computed per project.

One paragraph → a full spec

A worked example — illustrative output from a single description.

INPUT · NATURAL LANGUAGE

"A web app where students enroll in courses, submit assignments, and instructors grade them. Users sign in with email and password; admins manage accounts."

↓ 10-STAGE PIPELINE

FR-002 · High

The system shall allow students to enroll in available courses.

NFR-001 · Performance

Respond within 2 s for 95% of requests · metric tracked.

SR-001 · OWASP A01 Broken Access Control

Enforce role-based access control with least privilege.

TC-001 → covers FR-002

Enroll in a course → confirmation shown; course appears in "My courses".

Traceability

Built on a modern TypeScript stack

One language end-to-end — frontend, backend, and tooling.

FRONTEND

React 19

TypeScript

Tailwind CSS v4

Vite

React Router 7

BACKEND

Node.js

Express 4

TypeScript

JWT

Zod

Helmet

bcrypt

DATA & AI

PostgreSQL · Neon

Groq Llama-3.3-70b

Gemini 2.5-flash-lite

EXPORT & DEPLOY

pdfkit

docx.js

csv-writer

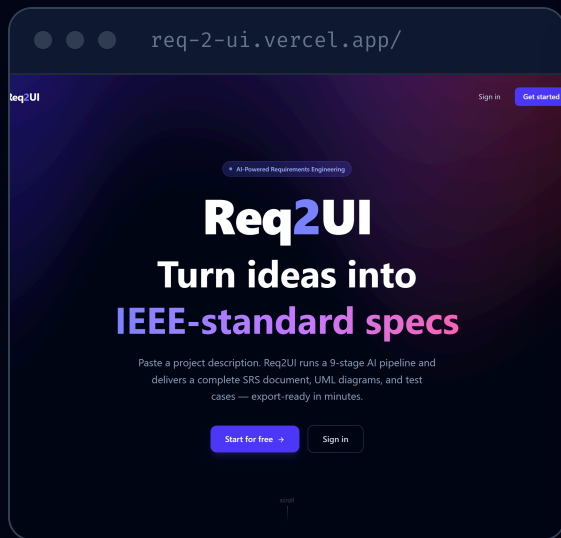
resvg-js

Vercel (FE)

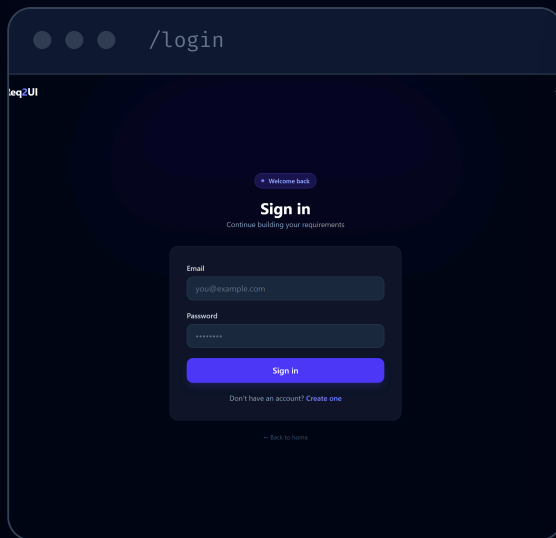
Render (BE)

The product, live

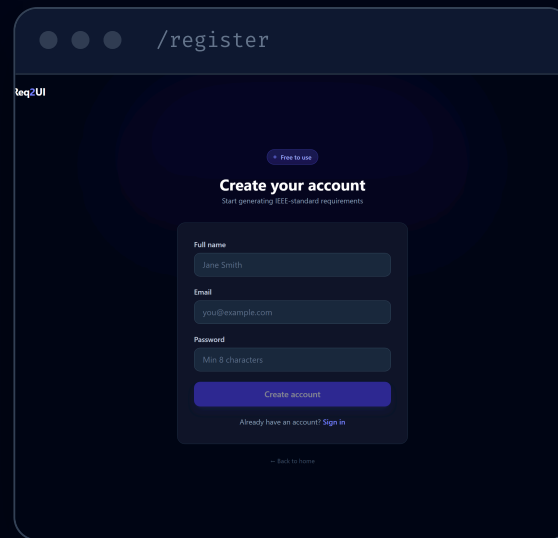
Deployed and running at req-2-ui.vercel.app — real screens.



Landing — the pitch in one screen.



Sign in — JWT auth, silent refresh.

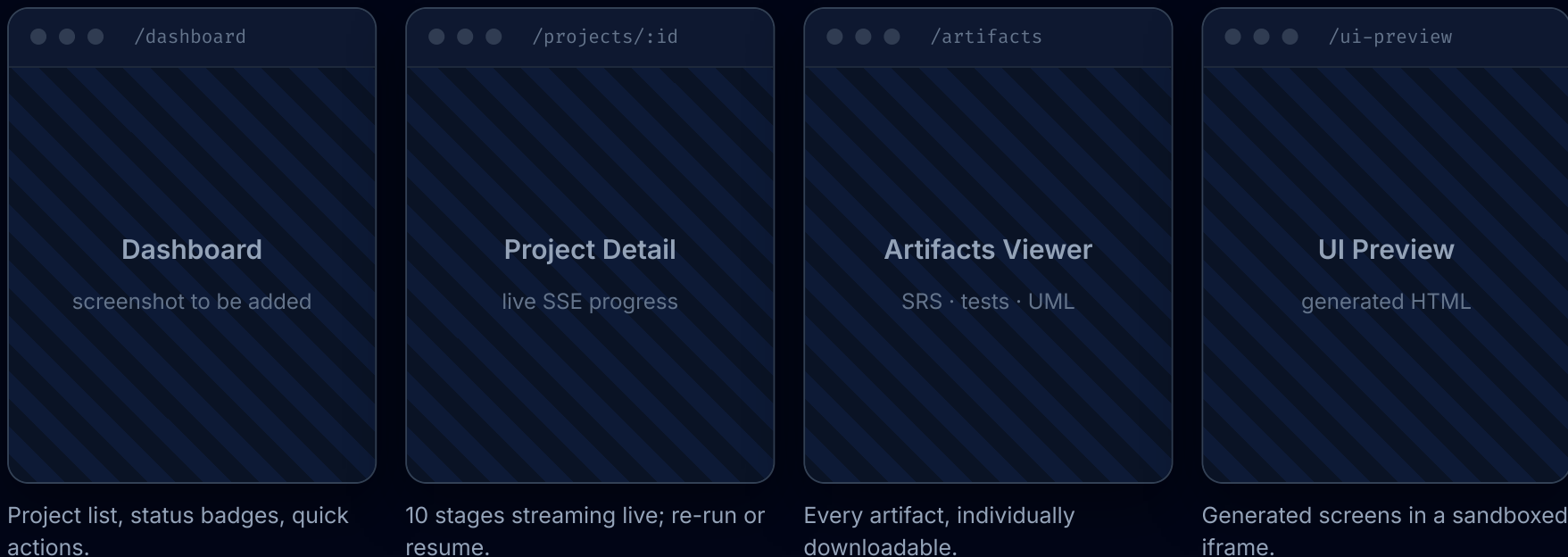


Register — bcrypt, Zod-validated.

Live and deployed: Vercel (frontend) + Render (backend) + Neon (database).

Inside the app

The authenticated workflow — four screens we can walk through live.



Best seen live — a full walkthrough is available on request.

The AI writes runnable UI

Stage 10 — Gemini generates real HTML + Tailwind, not just descriptions.

Pass 1 · design system

A shared navbar, footer, Tailwind theme, and component language — so every screen is consistent.

Pass 2 · per-screen pages

One complete HTML page per *distinct* screen (deduplicated by category), all consuming the design system.

Result

Low-fidelity but **runnable** — opens straight in a browser; great for early stakeholder feedback.

REPRESENTATIVE GENERATED OUTPUT

```
<nav class="bg-slate-900 border-b border-slate-800">
  <div class="flex items-center justify-between px-6 h-14">
    <span class="font-bold text-indigo-400">CourseHub</span>
    <a class="btn-primary" href="/enroll">Enroll</a>
  </div>
</nav>
<main class="grid gap-4 p-8 md:grid-cols-3">
  <article class="rounded-xl bg-slate-900 p-5">
    <h3 class="font-semibold">Intro to Databases</h3>
    <p class="text-slate-400">12 lessons · 4 assignments</p>
    <button class="mt-3 btn-primary">Enroll now</button>
  </article>
</main>
```

Validated, not just demoed

Automated tests + standards conformance across every artifact type.

24

AUTOMATED TESTS PASSING

Across 2 suites: auth routes + export service.

IEEE 830

Generated SRS follows IEEE 830 structure.

IEEE 829

Test cases use IEEE 829 fields.

OWASP Top 10 (2021)

Every SR linked to a Top-10 category.

IEEE 12207

Process standard followed end-to-end.

UML

Use-case, class, sequence per the UML spec.

Req2UI by the numbers

One paragraph in — a whole spec phase out.

10

stages

end-to-end AI pipeline.

10

artifact types

reqs, tests, UML, UI, traceability.

2

LLMs

Groq for text, Gemini for code.

10

OWASP cats

Top 10 (2021) mapped in stage 4.

4

export formats

PDF · DOCX · LaTeX · CSV.

24

tests passing

across auth + export suites.

Three IEEE standards (830, 829, 12207) · resumable, concurrent runs · live streamed to the browser.

Manual spec phase vs. Req2UI

Same deliverables — a fraction of the effort.

DIMENSION	MANUAL / TODAY'S TOOLS	REQ2UI
Time to first draft	Days to weeks (typical)	Minutes
Consistency	Varies by author	Standardised, IEEE-structured
Security requirements	Often skipped / late	OWASP Top 10, by default
Test cases	Separate effort	Generated (functional + security)
UI prototype	Different tool, no link to spec	Generated, requirement-driven
Traceability	Manual or missing	Automatic, end-to-end

Time figures are typical/illustrative, not a controlled benchmark.

What we delivered

Five objectives, five outcomes — all met.

01 **Single input** — accepts free-form plain-language descriptions.

Met

02 **Full SRS** — IEEE 830 with functional, non-functional & security requirements.

Met

03 **Test cases** — functional (IEEE 829) + security (OWASP-driven).

Met

04 **UML + UI** — Mermaid diagrams + HTML/Tailwind per screen.

Met

05 **Traceability + exports** — coverage matrix + PDF/DOCX/LaTeX/CSV.

Met

Honest limits, clear roadmap

What's bounded — and where we go next.

LIMITATIONS

- Output quality tracks the LLM and the detail of the input.
- Throughput is bounded by free-tier API rate limits (Grok & Gemini).
- Generated UI is a low-fidelity prototype — not production markup.
- Some NFR targets (load, full WCAG) need dedicated test runs.
- OWASP coverage is Top-10-only; ASVS depth is future work.

FUTURE WORK

- Admin dashboard for project + user management.
- In-app artifact editing and refinement.
- Self-critique refinement loops (LLM reviews its own output).
- More export targets (Markdown, OpenAPI).
- Domain fine-tuning / retrieval augmentation; ASVS-level security.
- Auto-run generated tests against the generated UI.

CONCLUSION

Req2UI closes the gap.

The first integrated pipeline covering natural language → SRS → UI prototype → functional + security test cases — with automatic security generation and full end-to-end traceability.

ONE INPUT · TEN STAGES · A COMPLETE, SECURITY-AWARE ARTIFACT SUITE

IEEE 830

IEEE 829

OWASP Top 10 (2021)

References & standards

Grounded in the literature and recognised standards.

SELECTED REFERENCES

1. O. Matsuura et al. — *Automatic generation of web-based UI prototypes from UML requirements models.*
2. R. Khankhoje — *An In-Depth Review of Test Automation Frameworks: Types and Trade-offs.*
3. B. Wei (2023) — *Requirements Are All You Need: From Requirements to Code with LLMs.*
4. S. K. T. Pushparaj et al. — *A Survey on Automatic Test Case Generation.*
5. A. Fatima et al. (2025) — *NLP-based generation of software test cases.*
6. A. Goknil et al. (2015) — *Automatic Generation of System Test Cases from Use Case Specifications.*
7. R. Batool et al. (2025) — *Automated Categorization of Software Security Requirements.*

STANDARDS APPLIED

IEEE 830

IEEE 829

IEEE 12207

OWASP Top 10 (2021)

UML

WCAG 2.1 AA

BUILT WITH

React 19

Express

Neon Postgres

Groq

Gemini

Slidev

Thank you.

Questions are welcome — on any stage, decision, or trade-off.

TEAM Mennatallah Hany · Mayar Hany · Yasmin Haitham · Malak Tarek · Mohamed Bassem

SUPERVISOR Dr. Salma Radwan Abdel-Hamid

Arab Academy for Science, Technology & Maritime Transport — 2025/2026